

Desafio Prático: Financy

Conheça o projeto

Vamos desenvolver uma aplicação FullStack de gerenciamento de finanças: o Financy!

O objetivo é criar uma aplicação que permita a organização de finanças, com gestão de transações e categorias.

Instruções

Estrutura, regras e requisitos do projeto

Descrição e requisitos funcionais do Back-end

Nesse projeto back-end, será desenvolvido uma API para gerenciar a organização das finanças.

Funcionalidades e Regras

- O usuário pode criar uma conta e fazer login
- O usuário pode ver e gerenciar apenas as transações e categorias criadas por ele
- Deve ser possível criar uma transação
- Deve ser possível deletar uma transação
- Deve ser possível editar uma transação
- Deve ser possível listar todas as transações
- Deve ser possível criar uma categoria
- Deve ser possível deletar uma categoria
- Deve ser possível editar uma categoria
- Deve ser possível listar todas as categorias

⚠ Para esse desafio é esperado que você utilize o banco de dados SQLite. Caso prefira pode usar o Postgres como opção.

Variáveis ambiente

Todo projeto tem diversas configurações de variáveis que devem ser diferentes de acordo com o ambiente que ele é executado. Para isso, importante sabermos, de forma fácil e intuitiva, quais variáveis são essas. Então é obrigatório que esse projeto tenha um arquivo `.env.example` com as chaves necessárias.

`JWT_SECRET=`

`DATABASE_URL=`

Caso adicione variáveis adicionais, lembre-se de incluí-las no `.env.example`.

Requisitos Não Funcionais

É obrigatório o uso de:

- TypeScript
- GraphQL
- Prisma
- SQLite

Boas Práticas

- Não se esqueça de habilitar o CORS na aplicação.

Quer ir além?

Lembre-se de seguir todos os requisitos obrigatórios, principalmente os relacionados às tecnologias, como o GraphQL. Mas você aprendeu bastante conteúdo ao longo da sua jornada da Pós-Graduação e, se quiser se desafiar, pode aplicar os conceitos de boas práticas e DevOps que estudou. Apenas lembre-se de configurar todo o ambiente para que a aplicação possa ser executada localmente.

Descrição e requisitos funcionais do Front-end

Nesse projeto front-end será desenvolvido uma aplicação React que, em conjunto com a API, permite o gerenciamento de transações e categorias.

Funcionalidades e Regras

Assim como na API, temos as seguintes funcionalidades e regras:

- O usuário pode criar uma conta e fazer login
- O usuário pode ver e gerenciar apenas as transações e categorias criadas por ele
- Deve ser possível criar uma transação
- Deve ser possível deletar uma transação
- Deve ser possível editar uma transação
- Deve ser possível listar todas as transações
- Deve ser possível criar uma categoria
- Deve ser possível deletar uma categoria
- Deve ser possível editar uma categoria
- Deve ser possível listar todas as categorias

Além disso, também temos algumas regras importantes específicas para o front-end:

- É obrigatória a criação de uma aplicação React usando GraphQL para consultas na API e Vite como bundler;
- Siga o mais fielmente possível o layout do Figma;

Páginas

Essa aplicação possui 6 páginas e dois modais com os formulários (Dialog):

- A página raiz (/) que exibe:
 - Tela de login caso o usuário esteja deslogado
 - Tela dashboard caso usuário esteja logado

Variáveis ambiente

Todo projeto tem diversas configurações de variáveis que devem ser diferentes de acordo com o ambiente que ele é executado. Para isso, importante sabermos, de forma fácil e intuitiva, quais variáveis são essas. Então é obrigatório que esse projeto tenha um arquivo `.env.example` com as chaves necessárias:

`VITE_BACKEND_URL=`

Boas Práticas

- Comece o projeto pela aba Style Guide no Figma. Dessa forma, você prepara todo o seu tema, fontes e componentes e quando for criar as páginas vai ser bem mais tranquilo;
- Assim com a experiência do usuário é importante (UX), a sua experiência no desenvolvimento (DX) também é muito importante. Por isso, apesar de ser possível criar essa aplicação sem nenhuma biblioteca, recomendamos utilizar algumas bibliotecas que vão facilitar tanto o desenvolvimento inicial quanto a manutenção do código;

Requisitos Não Funcionais

É obrigatório o uso de:

- Typescript
- React
- Vite sem framework
- GraphQL

É flexível o uso de:

- TailwindCSS
- Shadcn
- React Query
- React Hook Form
- Zod

Quer ir além?

Se você quer se desafiar e explorar conceitos além do que foram propostos no desafio, temos algumas ideias em mente para te inspirar:

⚠️ A correção do seu desafio é apenas levando em conta as regras e funcionalidades obrigatórias mencionadas nas etapas anteriores. Portanto, envie o seu código para correção antes de implementar novas funcionalidades (ou crie uma nova branch com o seu código alterado)

- Upload de imagem: o upload de imagem para o avatar do usuário é opcional, mas que tal você usar o que aprendeu e se desafiar adicionando a imagem de perfil do usuário.

Entrega

Esse desafio deve ser entregue na nossa plataforma. Para o envio, é necessário criar um repositório no GitHub e enviar o link do seu repositório na nossa plataforma com a sua resolução!

Porém, é **importante seguir alguns padrões** para que possamos **corrigir** corretamente o seu projeto:

- O repositório deve estar público;
- O repositório deve conter a resolução do desafio;
- O repositório deve ter duas subpastas
 - backend vai conter a resolução completa do desafio Back-end;
 - frontend vai conter a resolução completa do desafio Front-end.
- O repositório deve conter o código referente as regras e funcionalidades obrigatórias. Caso queira se desafiar com funcionalidades extras, crie o código com essas alterações em uma nova branch, preservando o seu código original do desafio.

Após concluir o desafio, se você se sentir confortável, o que acha de postar no LinkedIn contando como foi a sua experiência compartilhando o seu projeto e o seu aprendizado? É uma excelente forma de demonstrar seus conhecimentos e atrair novas oportunidades! 🧐

E pode marcar a gente, viu? Vai ser incrível acompanhar toda a sua evolução! 💜